

Azure Kubernetes Microservices Project Documentation

Project Overview

This project involved building and deploying a cloud-native Social Media application using a microservices architecture. The goal was to gain hands-on experience with Docker, Kubernetes, Azure services, Azure Container Registry (ACR), and container orchestration.

The application includes:

- User authentication
- User profile and bio
- Posts
- Comments
- Media upload
- Azure SQL Database integration
- Azure Blob Storage integration

The project was designed to be Kubernetes-ready and focused on learning real-world DevOps concepts.

Architecture

User ↓ Ingress ↓ Frontend Service ↓

Auth Service Post Service Comment Service Media Service

↓ Azure SQL Database Azure Blob Storage

Microservices Created

Frontend Service

Responsibilities:

- Login page
- Registration page

- Feed page
 - Profile page
 - Create post page
-

Auth Service

Responsibilities:

- Register users
- Login users
- Generate JWT tokens
- Profile information

Endpoints:

- POST /register
 - POST /login
 - GET /profile
-

Post Service

Responsibilities:

- Create posts
 - View posts
 - Delete posts
 - Like posts
-

Comment Service

Responsibilities:

- Add comments
 - View comments
 - Delete comments
-

Media Service

Responsibilities:

- Upload images
- Delete images

- Store files in Azure Blob Storage
-

Technologies Used

Backend:

- Flask
- SQLAlchemy
- Gunicorn
- PyODBC

Cloud:

- Azure SQL Database
- Azure Blob Storage
- Azure Container Registry

Containerization:

- Docker
- Docker Compose

Orchestration:

- Kubernetes

Infrastructure:

- Kubernetes YAML

CI/CD:

- GitHub Actions
-

Local Development

Run application using Docker Compose:

```
docker compose up --build
```

This created:

- frontend container

- auth-service container
- post-service container
- comment-service container
- media-service container

Docker Compose automatically created networking between services.

Kubernetes Setup

Enabled Kubernetes in Docker Desktop.

Verification:

```
kubectl get nodes
```

Expected:

```
docker-desktop Ready
```

Kubernetes Resources Used

Created:

- Deployments
 - Services
 - ConfigMaps
 - Secrets
 - Ingress
-

Deployment Files

Folder structure:

kubernetes/

- auth-deployment.yaml
- post-deployment.yaml
- comment-deployment.yaml

- media-deployment.yaml
 - frontend-deployment.yaml
 - services.yaml
 - configmap.yaml
 - secrets.yaml
 - ingress.yaml
-

ConfigMap

Purpose:

Store non-sensitive configuration.

Examples:

- DB server name
- Service URLs
- Blob container names

Applied using:

```
kubectl apply -f configmap.yaml
```

Secrets

Purpose:

Store sensitive values.

Examples:

- SQL password
- JWT secret
- Storage connection strings

Important:

Used stringData instead of base64 encoded data.

Applied using:

```
kubectl apply -f secrets.yaml
```

Kubernetes Services

Purpose:

Provide stable communication between pods.

Examples:

`http://auth-service http://post-service http://comment-service http://media-service`

Type used:

ClusterIP

Ingress Configuration

Purpose:

Route incoming traffic.

Routes:

`/ → frontend /api/auth → auth-service /api/posts → post-service /api/comments → comment-service /api/media → media-service`

Issue encountered:

Regex paths:

`/api/auth/?(.*)`

failed with Prefix pathType.

Resolution:

Used:

`/api/auth`

without regex.

Azure Container Registry

Created Azure Container Registry for image storage.

Images stored:

- auth-service
- post-service
- comment-service
- media-service
- frontend

Built images using cloud builds:

```
az acr build --registry YOUR_ACR --image auth-service:v1 ./auth-service
```

Repeated for all services.

Verify repositories:

```
az acr repository list --name YOUR_ACR --output table
```

AKS Cluster Connection

Connected local kubectl context to AKS:

```
az aks get-credentials --resource-group SocialLiteRG --name SocialLiteCluster
```

This command downloads cluster credentials and updates kubeconfig so kubectl commands target the AKS cluster.

Verify:

```
kubectl get nodes
```

Update Kubernetes Deployments for ACR

Deployment files initially used local images:

```
image: auth-service:latest
```

Updated to Azure Container Registry:

```
image: <your-acr-name>.azurecr.io/auth-service:latest
```

Updated all deployment files:

- auth-deployment.yaml
- post-deployment.yaml
- comment-deployment.yaml
- media-deployment.yaml
- frontend-deployment.yaml

Reason:

AKS cannot access local Docker images and must pull images from ACR.

Install NGINX Ingress Controller on AKS

AKS does not include an Ingress Controller by default.

Installed NGINX Ingress Controller:

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.12.2/deploy/static/provider/cloud/deploy.yaml
```

This creates:

- NGINX Ingress Controller
- Azure Load Balancer
- Public IP

Wait for provisioning:


```
kubectl get service ingress-nginx-controller --namespace=ingress-nginx --watch
```

Once an external IP is assigned:

```
kubectl get ingress
```

Application traffic flow:

Internet ↓ Azure Public IP ↓ Azure Load Balancer ↓ NGINX Ingress Controller ↓ Ingress Rules ↓ Services
↓ Pods

Kubernetes Deployment

Applied:

```
kubectl apply -f kubernetes/
```

Checked:

```
kubectl get pods  
kubectl get svc  
kubectl get ingress
```

Troubleshooting Performed

Pending Pods

Reason:

Resource constraints.

Fix:

Reduced replicas:

```
replicas: 1
```

CrashLoopBackOff

Issue:

Application repeatedly crashed.

Investigated using:

```
kubectl logs POD_NAME
```

Found:

```
ImportError: libodbc.so.2
```

Reason:

Linux ODBC dependencies missing.

Resolution:

Updated Dockerfile to install:

- unixodbc
- unixodbc-dev
- msodbcsql18

Rebuilt image:

```
az acr build --registry YOUR_ACR --image auth-service:v2 ./auth-service
```

Redeployed.

Useful Commands

Pods:

```
kubectl get pods
```

Logs:

```
kubectl logs POD_NAME
```

Describe:

```
kubectl describe pod POD_NAME
```

Restart deployment:

```
kubectl rollout restart deployment auth-service
```

Scale:

```
kubectl scale deployment auth-service --replicas=3
```

Learning Outcomes

This project provided hands-on experience with:

- Docker containers
 - Microservices
 - Azure services
 - Azure Container Registry
 - Kubernetes Deployments
 - Services
 - Ingress
 - ConfigMaps
 - Secrets
 - Pod troubleshooting
 - CrashLoopBackOff debugging
 - Cloud-native architecture
 - Container orchestration
-

Future Improvements

- Deploy to AKS
- Add Horizontal Pod Autoscaler
- Implement Helm charts
- Add Prometheus and Grafana monitoring
- Implement CI/CD with GitHub Actions
- Add Azure Key Vault
- Add Managed Identity